

Actividad de Evaluación N° 1 (AE1)

Desarrollo Incremental de API RESTful, Informe Técnico y Control de Versiones

Institución: Universidad Cuenca del Plata (UCP) – Sede Posadas

Facultad: Facultad de Ingeniería y Tecnología (FAITA)

Carrera: Ingeniería en Sistemas de Información (3er Año)

Asignatura: Paradigmas y Lenguajes de Programación III (RR N° 161/15)

Docente: Prof. Carlos Emiliano Pereyra.

Ciclo Lectivo: 2026 – 2° Cuatrimestre

Fechas Clave y Modalidad de Entrega

- **Fecha de Lanzamiento / Publicación:** 19 de Agosto de 2026
- **Fecha Límite de Entrega Digital (Moodle):** 02 de Septiembre de 2026 (18:00 hs)
- **Jornada de Exposición y Control en Laboratorio:** 02/09/2026 y 03/09/2026
- **Modalidad de Trabajo:** En parejas (máximo 2 integrantes, coincidentes con el equipo asignado para el TIFAS / AE4).

1. Objetivos de Aprendizaje

1. Diseñar y codificar una primera versión **funcional e incremental** de una API RESTful aplicando la semántica del protocolo **HTTP/HTTPS** (verbos, encabezados, cuerpos en JSON y familias de códigos de estado).
2. Estructurar el proyecto de software mediante un patrón de diseño en capas (separación de rutas, controladores y modelos/recursos).
3. Implementar mecanismos de validación de datos de entrada y **middleware de manejo centralizado de errores** (400 Bad Request, 404 Not Found, 500 Internal Server Error).
4. Redactar un **Informe Técnico (Bitácora de Avance)** que documente el problema, los requerimientos, la arquitectura y la fundamentación del stack tecnológico seleccionado.
5. Gestionar el ciclo de vida del desarrollo utilizando **Git y GitHub** (commits progresivos, ramas y documentación en [README.md](#)).
6. Demostrar habilidades de comunicación técnica mediante la defensa oral y prueba en

vivo (*live testing*).

2. Roadmap del Proyecto: El Paso a Paso Incremental (Clase)

El desarrollo del **AE1** constituye el **Hito 1 obligatorio** del Proyecto Integrador Final (TIFAS / AE4). La entrega debe seguir de forma estricta la secuencia metodológica planteada en la presentación del curso:

[Paso 1: Dominio] —>	[Paso 2: Entidades] —>	[Paso 3: GitHub] —>	[Paso 4: Parte Funcional] —>	[Paso 5: Entrega]
Definir problema (Ej: Clínica)	Modelar entidades (Ej: Turno, Médico)	Crear repositorio con nombre único	Codificar CRUD / API. de entidad principal	Informe PDF + Defensa

1. **Paso 1 - Definición del Dominio / Problema:** Elegir un sistema o app a desarrollar (ej. *Sistema de reserva de turnos para un taller mecánico, gestión de turnos para una clínica, catálogo e-commerce local, etc.*).
2. **Paso 2 - Definición de Entidades:** Modelar los componentes principales del dominio (ej. *Usuario, Turno, Vehículo, Servicio, Producto*).
3. **Paso 3 - Creación del Repositorio en GitHub:** Crear un repositorio público dedicado con el nombre representativo del proyecto (ej. *sistema-turnos-taller* o *api-gestion-clinica*).
4. **Paso 4 - Desarrollo de la Parte Funcional (Requisito AE1):** Desarrollar un **CRUD funcional y operativo** conectado al backend mediante una API RESTful para al menos la **entidad principal** del sistema.
5. **Paso 5 - Subida de Código e Informe:** Registrar el historial de *commits*, redactar el informe técnico (PDF) y presentar la solución en Moodle y en el laboratorio.

3. Requerimientos Técnicos del Software

A. Endpoints de la API RESTful (CRUD Funcional de la Entidad Principal)

El servicio backend debe exponer al menos **4 endpoints** operativos y probados para la entidad núcleo del proyecto:

1. **GET /api/v1/[recurso]**: Obtener el listado completo de elementos (ej. todos los turnos o productos). Retorna **200 OK** con un payload JSON.
2. **GET /api/v1/[recurso]/{id}**: Obtener un elemento por su ID.
 - Si existe —> Retorna **200 OK** y el objeto JSON.
 - Si no existe —> Retorna **404 Not Found** con estructura de error estandarizada.
3. **POST /api/v1/[recurso]**: Crear un nuevo registro.
 - Valida campos obligatorios en el payload JSON.
 - Si es correcto —> Agrega el elemento y retorna **201 Created**.
 - Si faltan campos —> Retorna **400 Bad Request** indicando la falla explícita.
4. **PUT /api/v1/[recurso]/{id}** o **DELETE /api/v1/[recurso]/{id}**: Modificar o eliminar un recurso existente retornando **200 OK** o **204 No Content**.

B. Arquitectura, Stack Tecnológico y Calidad de Código

- **Stack Tecnológico (LIBRE ELECCIÓN)**: Cada grupo selecciona de forma autónoma las tecnologías a utilizar:
 - **Lenguaje**: Libre elección (Python, JavaScript/TypeScript, Java, C#, PHP, Go, etc.).
 - **Framework / Módulo Web**: Libre elección (FastAPI, Flask, Express.js, NestJS, Spring Boot, ASP.NET Core, Laravel, u otros).
 - **Persistencia**: Libre elección (PostgreSQL, MySQL, SQLite, MongoDB, o estructuras persistentes en archivos/memoria para este primer hito).
- **Entorno de Trabajo Recomendado**: **VS Code** (o IDE profesional equivalente) + Terminal de Comandos.
- **Estructura en Capas**: Código organizado en módulos separados (Rutas, Controladores/Servicios, Datos). No se aceptan archivos monolíticos.
- **Middleware de Errores**: Capturador centralizado que devuelva un JSON estructurado (**500 Internal Server Error**) ante fallos imprevistos.
- **Variables de Entorno**: Uso de archivo **.env** (puerto, modo de ejecución, credenciales).

4. Requerimientos del Informe Técnico y Bitácora (PDF)

El grupo deberá adjuntar un **Informe Técnico y Bitácora de Avance** en PDF. El cuerpo principal consta de **máximo 2 páginas de texto técnico** (excluyendo carátula, bibliografía y anexos visuales).

Apartados Obligatorios del Informe:

1. **Introducción y Problema Abordado:** Descripción del contexto y justificación de la solución propuesta (Paso 1 del Roadmap).
2. **Modelo de Dominio y Entidades:** Descripción de las entidades del sistema y la entidad principal seleccionada para el AE1 (Paso 2 del Roadmap).
3. **Metodología de Trabajo:** Flujo de trabajo ágil e incremental en equipo.
4. **Justificación del Stack Tecnológico:** Argumentación técnica de la elección del Lenguaje, Framework y Persistencia.
5. **Requerimientos del Sistema:**
 - *Requerimientos Funcionales (RF):* Mínimo 4 RF.
 - *Requerimientos No Funcionales (RNF):* Mínimo 4 RNF (ej. tiempo de respuesta, formato JSON, disponibilidad).
6. **Diagramas Arquitectónicos (si corresponde):** Diagrama de Capas, Entidad-Relación o Mapa de Rutas.
7. **Especificación y Fundamentación Algorítmica:** Justificación del modelo de ejecución (**Asíncrono / No Bloqueante o Concurrente/Multihilo**).
8. **Conclusión y Próximos Pasos (AE2/TIFAS):** Breve síntesis del alcance funcional obtenido y la hoja de ruta para la siguiente etapa.
9. **Anexo - Bitácora Visual y Repositorio (Página adicional):**
 - Enlace al **Repositorio Público de GitHub** (Paso 3 del Roadmap).
 - Capturas de pantalla del *live testing* en Postman / Thunder Client / Terminal (Paso 4 del Roadmap).
 - Captura del historial de *commits* semánticos demostrando la colaboración de ambos integrantes (Paso 5 del Roadmap).

5. Dinámica de Exposición y Defensa Oral (02/09 y 03/09)

En el laboratorio de informática:

1. **Demostración en Vivo (5 minutos):** La pareja ejecutará su servidor local desde la terminal y realizará pruebas en tiempo real consumiendo los endpoints del CRUD funcional.
2. **Inspección de Código e Informe (5 minutos):** El docente revisará el informe y solicitará abrir módulos de código específicos en VS Code para verificar la arquitectura en capas y las validaciones.

3. **Preguntas de Control Individual (5 minutos):** Preguntas individuales a cada integrante sobre distintos aspectos acerca del trabajo.



6. Rúbrica de Evaluación Institucional (UCP / FAITA)

La calificación final de la **AE1** se calcula en escala de **1 a 10**:

Dimensión Evaluable	Criterio de Excelencia (100%)	En Desarrollo (60%-80%)	Requiere Mejora (0%-50%)	Ponderación
Saber (Teoría HTTP, REST & Asincronía)	Aplica de forma impecable la semántica HTTP, verbos, headers, JSON y status codes. Fundamenta el stack elegido y responde preguntas de control oral.	Presenta imprecisiones menores en los códigos de estado, en la justificación de su stack o en el modelo asíncrono/concurrente.	Confunde verbos HTTP o no puede fundamentar las decisiones tecnológicas tomadas durante el coloquio.	25%
Saber Hacer (Parte Funcional, API & GitHub)	CRUD funcional operativo de la entidad principal con arquitectura en capas, validaciones, middleware de error y repositorio Git con commits semánticos de ambos miembros.	La API funciona pero carece de arquitectura en capas, faltan validaciones de entrada o los commits son de un solo integrante.	El servidor no ejecuta, la parte funcional no responde a las peticiones o no se utiliza control de versiones Git.	45%

Saber Hacer (Informe Técnico & Bitácora)	Informe de máx. 2 páginas estructurado con todos los apartados (Dominio, Entidades, Stack, Concurrencia, Diagramas y Bitácora de anexos).	El informe está incompleto, carece de la fundamentación técnica del stack o excede el límite de páginas.	No presenta el informe técnico o presenta un texto genérico sin relación con el proyecto desarrollado.	20%
Saber Ser (Profesionalismo & Defensa)	Entrega puntual, excelente fluidez técnica en el <i>live testing</i> , trabajo en equipo equilibrado y vocabulario técnico específico.	Depende del docente para resolver bloqueos simples en el testing o la participación en la defensa es desequilibrada.	Presentación fuera de término, incapacidad de ejecutar la prueba en vivo o falta de compromiso.	10%



7. Entregables en Moodle

Cada pareja deberá subir al buzón habilitado en Moodle un **único archivo PDF** titulado **AE1_Grupox_[NombreCompletoIntegrantesGrupo]** que contenga:

1. Carátula institucional UCP.
2. **Informe Técnico (2 páginas centrales).**
3. Conclusiones y Bibliografía.
4. **Anexo - Bitácora de Evidencias:** Enlace al Repositorio de GitHub y capturas del funcionamiento de la API.
5. **Presentación. (enlace o archivo)**